VibeMatch: Aesthetic-Driven Media Retrieval and Genre Classification

Team Members: Bowen Tan, Ethan Pan, Edoardo Mongardi, Owen Nie

1 Proposal

Problem Description

When people search for a movie or book, they usually type in a title, an author, or a genre tag. But often what someone really wants is a *vibe*—something like "a lonely journey through a neon-lit dystopian city"—and no traditional search engine handles that well. Our project tackles this by building a vibe-based retrieval system: given a free-form text description of a mood or atmosphere, we return movie posters and book covers whose visual style best matches it. We will use the **Kaggle Movie Poster** dataset ($\sim$7,000 images with genre labels) and the **Goodreads Book Cover** dataset ($\sim$20,000 images with genre metadata). This is an interesting problem because it goes beyond keyword matching into semantic understanding—the system has to learn what "neon-lit dystopian" actually *looks like*, which is a challenging cross-modal task that connects several topics from our course.

Methods

Our approach has three parts. First, to satisfy the course requirements, we build a CLIP-style **dual-encoder** system from scratch in PyTorch rather than calling an external API. We use pre-trained ResNet and DistilBERT as frozen feature extractors and connect them through custom-trained **projection layers** that map both modalities into a shared latent space. We train these projection layers with a **symmetric contrastive loss**, which demonstrates core syllabus topics like iterative optimization and gradient-based learning. Contrastive learning is a good fit because we don't have fine-grained labels—just genre tags—and it can learn from that kind of weak supervision. Second, at query time we encode the user's text and find the closest image embeddings using **cosine similarity**, returning the top-k results. This is essentially a **nearest-neighbor search** in the learned space. Third, we train a **MLP genre classifier** on the frozen image embeddings to predict genre. Rather than using this only as an offline evaluation, we integrate it directly into the web app: each search result is displayed with auto-generated genre tags underneath, giving users extra context about the retrieved media. For evaluation, we use a **genre-stratified split** instead of random splitting so that we can test whether the model generalizes to unseen visual styles, rather than just memorizing specific covers.

2 Design Doc

2.1 Repo Structure

```
vibematch/
|-- README.md                # Project overview, setup, and usage instructions
|-- requirements.txt         # Python dependencies
|-- app.py                   # Streamlit entry point: launches the web app
|-- configs/
|   |-- train_config.yaml    # Training hyperparameters
|   +-- app_config.yaml      # App settings (top-k, index path, model weight path)
|-- scripts/
|   |-- train_clip.py        # Trains projection layers with contrastive loss
|   |-- train_classifier.py  # Trains the MLP genre classifier on frozen embeddings
|   +-- build_index.py       # Generates the FAISS index from image embeddings
|-- src/
|   |-- loaders/
|   |   |-- dataset.py       # Dataset classes and image/text transforms
|   |   |-- data_loader.py    # DataLoader factory with batching and sampling logic
|   |   +-- split.py         # Genre-stratified train/val/test splitting logic
|   |-- model/
|   |   |-- encoder.py       # Frozen ResNet + DistilBERT w/ projection layers
|   |   |-- classifier.py    # MLP genre classifier for live genre tagging
|   |   +-- loss.py          # Symmetric contrastive loss function
|   +-- retrieval/
|       |-- engine.py        # FAISS index building and saving
|       +-- search.py        # Cosine-similarity query and top-k retrieval
|-- data/
|   |-- raw/                 # Original downloaded datasets (posters, covers)
|   +-- processed/           # Cleaned images and metadata CSVs
|-- models/                  # Saved checkpoints (.pt) and FAISS index (.bin)
|-- notebooks/
|   +-- exploration.ipynb    # EDA: dataset stats, sample visualizations
+-- tests/
    +-- test_pipeline.py     # Smoke tests for data loading, encoding, and retrieval
```

2.2 Division of Labor

- **Member A (Data & Evaluation):** Owns `src/loaders/` and `exploration.ipynb`. Handles downloading, cleaning, and preprocessing both datasets; implements genre-stratified train/val/test splits; produces dataset statistics and visualizations.
- **Member B (CLIP Encoder & Loss):** Owns `model/encoder.py`, `model/loss.py`, and `train_clip.py`. Implements the dual-encoder architecture using pre-trained ResNet and DistilBERT as frozen backbones with custom projection layers, implements the

symmetric contrastive loss, and runs the encoder training loop.

- **Member C (Classification & Testing):** Owns `classifier.py`, `train_classifier.py`, and `tests/`. Designs the MLP genre classifier on frozen embeddings, runs training/hyperparameter tuning, integrates the classifier into the web app to display auto-generated genre tags below each search result, and writes end-to-end tests.
- **Member D (Retrieval, Web App & Deployment):** Owns `retrieval/`, `build_index.py`, and `app.py`. Builds the FAISS index, develops and deploys the Streamlit web application, wires together the full query-to-results pipeline, manages the GitHub repo, and handles deployment (e.g., Streamlit Cloud).

2.3 Stub Code & GitHub

The public repository is at: `https://github.com/[Your-Username]/vibematch`. It currently contains a `README.md` with project description and setup instructions, a `requirements.txt` with all dependencies, and empty placeholder files for every module listed in the directory tree above (`data_loader.py`, `encoder.py`, `train_encoder.py`, `retrieval.py`, `classifier.py`, `app.py`, `exploration.ipynb`, `test_pipeline.py`).